

TrAFSP: Trace-Retrieval Augmented Few Shot Prompting for Link Generation

Nabin Chaulagain, Nan Niu, Sandeep Reddivari
University of North Florida
Jacksonville, FL, USA
{n01663977, nan.niu, sandeep.reddivari}@unf.edu

Jianzhang Zhang*
Hangzhou Normal University
Hangzhou, China
jianzhang.zhang@foxmail.com
* Corresponding author

Abstract—Software traceability is critical for impact analysis, compliance, and change management, but it is frequently neglected in practice due to the high cost of manual link creation. Existing automated approaches, including information retrieval, deep learning, and other LLM-based methods, have not achieved the accuracy and cost-efficiency needed for broad adoption. We propose TrAFSP (Trace-Retrieval Augmented Few-Shot Prompting), an approach that stores labeled source-target artifact pairs in a vector database and retrieves the most similar pairs at inference time as in-context demonstrations for an LLM. We compare TrAFSP against random few-shot prompting (FSP) and standard retrieval-augmented generation (RAG) across three publicly available traceability datasets using F_2 -score as the primary metric. TrAFSP achieves the highest F_2 -score on all three datasets, with statistically significant improvements over both baselines in most of the comparisons (bootstrap test, $N = 10000$, $\alpha = 0.05$), demonstrating that similarity-based retrieval of labeled traces provides more reliable and relevant context than random selection or unlabeled chunk retrieval.

Index Terms—requirements traceability, trace link recovery, requirements engineering, in-context learning

I. INTRODUCTION

Software traceability—the ability to identify and follow relationships between artifacts produced across the software development lifecycle [1]—plays a central role in ensuring software quality and maintainability. Trace links connect artifacts such as requirements, design documents, source code, and test cases, enabling stakeholders like developers and managers to perform impact analysis [2], [3], validate compliance with regulatory standards [4], [5], and manage changes across large, evolving systems [6], [7]. In safety-critical domains such as aerospace engineering, medical device engineering, and automobile engineering, maintaining complete and accurate trace links is often a requirement to meet compliance requirements [8].

Despite its importance, traceability is frequently ignored in practice because manually curating and maintaining trace links is expensive and time-consuming [1]. Automated approaches spanning information retrieval, machine learning, and deep learning have been explored [9]. Fine-tuning large language models (LLMs) has shown some strong results for trace link recovery [10], but these models tend to overfit on the specific projects on which they are trained, require substantial labeled data, and incur high computational costs [9], [11],

thereby limiting their practical adoption. Prompting-based LLM approaches that avoid fine-tuning entirely are therefore of practical interest [12]–[14].

Few-shot prompting (FSP) and retrieval-augmented generation (RAG) are two such approaches, but each has limitations. FSP often relies on randomly selected in-context examples, making performance sensitive to which examples happen to be chosen [15]. Standard RAG retrieves semantically similar document chunks but does not supply labeled demonstrations, leaving the model without direct guidance about what a correct link looks like.

In this paper, we propose TrAFSP (Trace-Retrieval Augmented Few-Shot Prompting), a novel approach which stores labeled source-target artifact pairs in a vector database and retrieves the most similar pairs at inference time as in-context demonstrations, addressing FSP’s sensitivity to random example selection. We also examine how performance scales as the labeled vector store grows, reflecting a realistic setting where labeled data becomes progressively available. Thus, our work contributes to innovative and practical ways of leveraging LLMs to tackle the software traceability challenges.

The rest of the paper is organized as follows. Section II discusses background and related work. Section III dives into the details of the three LLM-based approaches that we investigate, namely, FSP, standard RAG, and TrAFSP. We then present our empirical evaluations in Section IV, and finally conclude the paper in Section V.

II. RELATED WORK

Traditional information retrieval (IR) methods like TF-IDF and vector space models were some of the first automated approaches used for trace link recovery, but they mainly rely on matching similar words. Because of this, they struggle when related artifacts describe the same concept using different vocabulary [9], [16]. Supervised machine learning classifiers improve performance but require labeled data and do not generalize well across different datasets. Guo *et al.* [17] addressed the semantic gap with a deep learning approach that embeds artifact texts into shared semantic spaces, outperforming the IR baselines. Lin *et al.* [10] further advanced this direction with T-BERT, a BERT-based trace link recovery framework that uses transfer learning from related software engineering

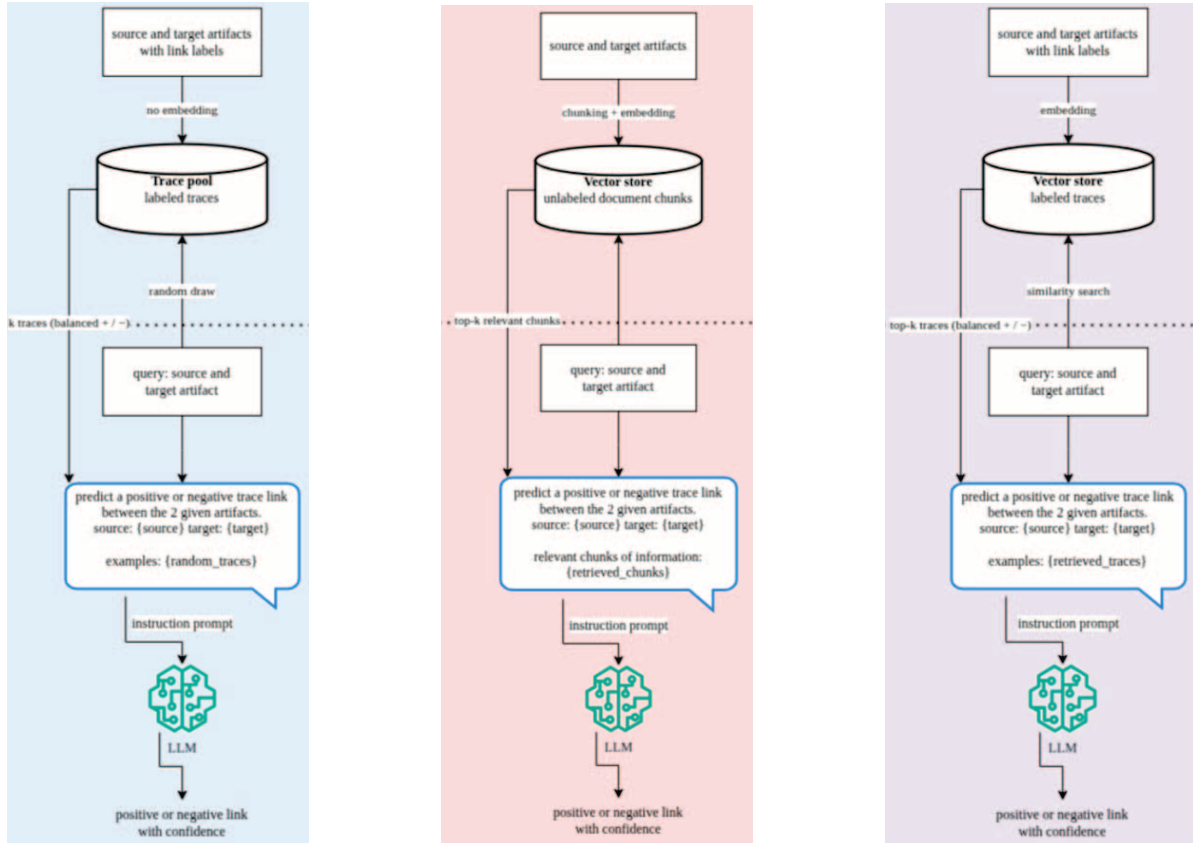


Fig. 1: Overview of FSP (left where k traces are retrieved in a balanced manner between positives and negative), Standard RAG (middle where top- k relevant chunks are retrieved), and TrAFSP (right where top- k traces are retrieved in a balanced manner between positives and negatives) across indexing (top of the dotted line) and inference (bottom of the dotted line) phases.

tasks to overcome data sparsity, achieving large gains over vector space models. Despite these advances, deep learning methods remain dependent on substantial training data and project-specific fine-tuning, limiting their usability in certain settings.

Alor *et al.* [18] showed that zero-shot LLM prompting outperforms IR-based methods for documentation-to-code traceability, establishing that LLMs possess sufficient domain knowledge from pretraining data alone for trace link recovery without task-specific training. Dahiya *et al.* [19] were among the first to explore traceability in a generative way, which inspired the work of Iluru *et al.* [20] to auto-generate functional descriptions for graphical user interfaces in a multi-modal environment. Rodriguez *et al.* [21] investigated prompt engineering strategies for LLM-based traceability, finding that prompt design significantly impacts link recovery accuracy and that more problem-specific prompts consistently outperform zero-shot baselines. Hey *et al.* [13] proposed a RAG-based approach combining zero-shot and chain-of-thought prompting, demonstrating strong results across several benchmarks and identifying few-shot prompting as a promising direction for future work. Fuchß *et al.* [22] introduced LiSSA, a generic RAG traceability framework in which chain-of-thought

prompting enables it to outperform task-specific methods. Both Hey *et al.* [13] and Fuchß *et al.* [22] retrieve individual artifact chunks rather than labeled pairs, leaving the model without explicit link supervision.

Trad and Chehab [11] demonstrated that retrieval-augmented few-shot prompting achieves results comparable to fine-tuning for code vulnerability detection at substantially lower resource cost. This motivates our work on TrAFSP to the traceability domain, introducing balanced class selection and varying the vector store size to study data efficiency.

III. METHODOLOGY

We frame trace link recovery as binary classification: given a source artifact and a target artifact, an LLM is prompted to predict, with reason and confidence, whether a trace link exists between them. The three approaches we compare differ only in how context is retrieved and presented to the model during inference; the underlying prompt structure and LLM are identical across all three.

A. Approaches

The three approaches that we investigate in this research are illustrated in Figure 1, and the prompt templates used for each are shown in Figure 2.

FSP	Standard RAG	TrAFSP
SYSTEM You are a classifier that determines whether a traceability link exists between a source and a target artifact. Both artifacts are from the same software system. Below are pairs of examples: Example 1: Source Artifact: {source_1} Target Artifact: {target_1} Label: 1 Example 2: Source Artifact: {source_2} Target Artifact: {target_2} Label: 0 (k randomly selected examples) USER Determine whether the following source and target artifacts are trace-linked. Source: {source} Target: {target} OUTPUT { label: 0 or 1, confidence: 0.0-1.0, reason: string }	SYSTEM You are a classifier that determines whether a traceability link exists between a source and a target artifact from the same software system. Relevant source context: [1] {source_chunk_1} [2] {source_chunk_2} Relevant target context: [1] {target_chunk_1} [2] {target_chunk_2} (top-n chunks retrieved by embedding similarity) USER Determine whether the following source and target artifacts are trace-linked. Source: {source} Target: {target} OUTPUT { label: 0 or 1, confidence: 0.0-1.0, reason: string }	SYSTEM You are a classifier that determines whether a traceability link exists between a source and a target artifact. Both artifacts are from the same software system. Below are pairs of examples: Example 1: Source Artifact: {source_1} Target Artifact: {target_1} Label: 1 Example 2: Source Artifact: {source_2} Target Artifact: {target_2} Label: 0 (k examples retrieved via cosine similarity) USER Determine whether the following source and target artifacts are trace-linked. Source: {source} Target: {target} OUTPUT { label: 0 or 1, confidence: 0.0-1.0, reason: string }

Fig. 2: Prompt templates for FSP, Standard RAG, and TrAFSP. The key difference lies in the SYSTEM section: FSP injects randomly selected labeled pairs, Standard RAG injects retrieved document chunks without labels, and TrAFSP injects labeled pairs retrieved via cosine similarity.

TABLE I: Traceability Datasets

Dataset	Source	Target	Total Pairs	Positive Links
EasyClinic	113 Mixed artifacts	160 Mixed artifacts	9,661	564
InfusionPump	124 Requirements	21 Components	2,604	131
CM1	22 High-level Requirements	53 Low-level Requirements	1,166	45

1) *TrAFSP*: During indexing, each training source-target pair together with its ground-truth label (linked or not linked) is embedded by concatenating the source and target texts and encoding them with a text embedding model. The resulting embedding and label are stored in a vector database. At inference, the query pair is embedded with the same model, and the top- k most similar training pairs are retrieved via cosine similarity search. These retrieved pairs, together with their ground-truth labels, are formatted as labeled in-context examples and prepended to the prompt before the query pair. The LLM is then asked to predict whether the query pair is linked.

2) *FSP*: The prompting structure is identical to TrAFSP. Rather than similarity-based retrieval, k examples are drawn uniformly at random from the training set. To ensure class balance, we always select $k/2$ positive (linked) and $k/2$ negative (not linked) examples. FSP isolates the effect of targeted retrieval from the benefit of having any labeled demonstrations at all.

3) *Standard RAG*: Source and target artifacts are chunked, embedded, and stored individually in a vector database, without label information. At inference, the top- k most similar

chunks to the query artifacts are retrieved and injected as plain context. No labeled examples are provided; the model must infer linkage from the retrieved context alone. Standard RAG represents a widely used prompting baseline that does not leverage labeled training data.

IV. EVALUATIONS

Our central research question (RQ) is: “*Is TrAFSP more effective than FSP and standard RAG for trace link recovery?*” To answer the RQ, we performed experiments on three publicly available traceability datasets. Table I displays some characteristics of the datasets.

Each dataset is split 70/30 for training and evaluation respectively. The training portion is used to build the vector store for TrAFSP or standard RAG and to draw random examples for FSP; the evaluation set is never seen during indexing.

FSP is evaluated at 0, 2, 4, 8, 16, 32, and 64 in-context examples to study how performance changes as more demonstrations are added. TrAFSP fixes the shot count at 4—preliminary experiments showed no consistent gain from additional shots—and instead varies the vector store size across 20%, 40%, 60%, 80%, and 100% of the training data, simulating a realistic

TABLE II: Bootstrap significance test results (F_2 -score, $N = 10000$, $\alpha = 0.05$). TrAFSP at each vector store size is compared against the best FSP and standard RAG configurations.

Dataset	Store	vs. Best FSP			vs. Best RAG		
		p	d	95% CI	p	d	95% CI
EasyClinic	@20%	<0.001	6.70	[0.066, 0.158]	<0.001	9.08	[0.140, 0.270]
	@40%	<0.001	8.70	[0.093, 0.180]	<0.001	10.61	[0.179, 0.301]
	@60%	<0.001	13.04	[0.138, 0.210]	<0.001	12.39	[0.208, 0.330]
	@80%	<0.001	11.69	[0.126, 0.198]	<0.001	11.99	[0.202, 0.321]
	@100%	<0.001	12.12	[0.130, 0.205]	<0.001	12.44	[0.205, 0.322]
InfusionPump	@20%	0.230	1.12	[-0.095, 0.207]	0.002	4.19	[0.078, 0.437]
	@40%	0.088	2.03	[-0.055, 0.236]	0.001	4.90	[0.140, 0.496]
	@60%	0.023	2.80	[0.003, 0.280]	<0.001	5.53	[0.160, 0.525]
	@80%	0.035	3.07	[-0.012, 0.277]	<0.001	5.85	[0.183, 0.529]
	@100%	0.009	3.81	[0.037, 0.322]	<0.001	6.45	[0.240, 0.565]
CM1	@20%	0.123	1.68	[-0.100, 0.337]	0.050	2.45	[-0.050, 0.472]
	@40%	0.022	3.19	[0.007, 0.424]	0.003	3.75	[0.095, 0.590]
	@60%	0.016	3.48	[0.026, 0.414]	0.006	3.82	[0.108, 0.599]
	@80%	0.113	1.90	[-0.087, 0.349]	0.026	2.65	[-0.005, 0.499]
	@100%	0.066	2.38	[-0.068, 0.398]	0.018	3.03	[0.031, 0.540]

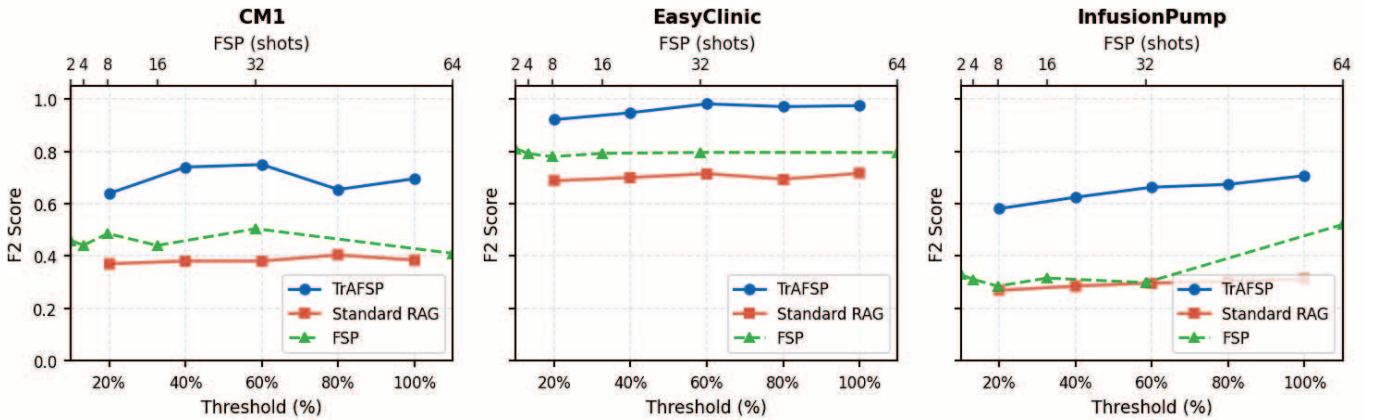


Fig. 3: F_2 -score comparison across datasets.

setting where labeled data becomes progressively available. Standard RAG uses 512-token chunks with 100-token overlap, a common configuration that balances information density against retrieval noise, and retrieves the top 10 chunks at inference.

For prompting we use Google’s `gemini-3-flash`, a cost-effective frontier model with strong instruction-following capabilities. Embeddings are generated with OpenAI’s `text-embedding-3-large` (3072 dimensions), which provides high-quality semantic representations suited for similarity-based retrieval. Temperature is set to 0 for all LLM API calls to remove randomness as a confounding variable.

All three approaches are evaluated on the same evaluation set using F_β -score ($\beta = 2$) as the primary metric. F_2 assigns twice as much weight to recall as to precision. In traceability, a missing link can propagate silently—leaving requirements unverified, breaking impact analysis chains, or causing compliance gaps in regulated systems [8]. A false positive, by contrast, only requires a manager or developer

to discard it, which carries comparatively little cost [23]. This cost asymmetry directly motivates a metric that weighs recall over precision. FSP and TrAFSP always select equal numbers of positive and negative examples (e.g., 4 shots = 2 positive + 2 negative) to reduce class imbalance bias, a refinement over Trad and Chehab [11]. Statistical significance is assessed using bootstrap hypothesis testing ($N = 10000$, $\alpha = 0.05$) with Cohen’s d as the effect size measure. Detailed statistical results are presented in Table II.

Figure 3 presents F_2 -scores for all approaches across the three datasets at each tested configuration. TrAFSP achieves the highest F_2 -score across all three datasets at every vector store size tested, answering our RQ positively.

On EasyClinic, TrAFSP reaches F_2 scores between 0.92 and 0.99 as the vector store grows, while FSP plateaus around 0.68–0.80 regardless of shot count and standard RAG stays flat near 0.69. All comparisons against both baselines are statistically significant ($p < 0.001$) with very large effect sizes—Cohen’s d ranges from 6.70 to 13.04 against FSP and from 9.08 to 12.44 against standard RAG (cf. Table II).

The consistent significance across all thresholds suggests that similarity-based retrieval provides a clear and reliable advantage on this dataset.

On InfusionPump, TrAFSP climbs steadily from 0.57 at 20% to 0.70 at 100% of the training data, while standard RAG remains flat near 0.26-0.31. TrAFSP significantly outperforms standard RAG at all thresholds ($p \leq 0.002$). The comparison against FSP is more nuanced: FSP starts low at few shots but rises sharply at higher shot counts, reaching around 0.52 at 64 shots. As a result, TrAFSP only reaches significance against FSP from 60% onward ($p = 0.023$), with the gap widening to $p = 0.009$ at 100%. This pattern suggests that on datasets with technical vocabulary, TrAFSP needs a sufficiently large vector store to consistently outpace a well-supplied FSP baseline.

On CM1, TrAFSP peaks at 0.75 at 60% before dipping to 0.65 at 80% and partially recovering to 0.69 at 100%. This non-monotonic pattern is notable and points to sensitivity in retrieval quality when the vector store contains fewer than the full training set on a small, sparse dataset (45 positive pairs out of 1,166 total). Statistical significance against both baselines is achieved at 40% and 60%, but is lost at 80% and 100% against FSP, where FSP’s random selection happens to draw competitive examples. Standard RAG remains the weakest baseline on CM1 throughout, with F_2 scores around 0.37-0.40.

FSP performance fluctuates across shot counts, likely due to sensitivity to random example selection. TrAFSP, by contrast, shows a more consistent upward trend as the vector store grows. Standard RAG consistently performs the worst across all datasets, indicating that retrieved document chunks are less effective than labeled demonstrations for trace link recovery—a specialized domain where, contrary to the findings of Min *et al.* [24], ground truth labels appear to provide essential task-specific supervision.

V. CONCLUSION

We have presented a novel TrAFSP approach to tackling trace link recovery. Experimental results show the superior performances of TrAFSP. Despite the promising results, a key limitation of TrAFSP is that it still requires a labeled training set to populate the vector store. Additionally, on both InfusionPump and CM1, absolute performance even at higher vector store sizes could be higher. We share our supplementary materials publicly at <https://zenodo.org/records/19451306>. This helps to facilitate replication and extensions [25], [26].

Future work could integrate TrAFSP with other prompting techniques, e.g., differential in-context learning [27], and put traceability in use such as refactoring [28] and reuse [29]. We are also interested in investigating strategies that require no labeled data while still providing acceptable results, e.g., unsupervised clustering methods [30], [31]. Semi-supervised approaches that combine a small labeled seed set with a large pool of unlabeled artifact pairs are a natural extension—they could bootstrap TrAFSP’s vector store with far less labeling effort. More targeted still, an active learning loop curated by LLMs themselves could select the most informative pairs for human labeling first, building an effective store iteratively

rather than requiring the entirety of the dataset to be labeled beforehand.

ACKNOWLEDGMENT

This work is partially supported by the Zhejiang Provincial Natural Science Foundation of China (Grant No. LQN26F020071).

REFERENCES

- [1] J. Cleland-Huang, O. Gotel, J. H. Hayes, P. Mäder, and A. Zisman, “Software traceability: Trends and future directions,” in *Future of Software Engineering (FOSE)*, Hyderabad, India, May-June 2014, pp. 55–69. [Online]. Available: <https://doi.org/10.1145/2593882.2593891>
- [2] W. Wang, A. Gupta, N. Niu, L. D. Xu, J.-R. C. Cheng, and Z. Niu, “Automatically tracing dependability requirements via term-based relevance feedback,” *IEEE Transactions on Industrial Informatics*, vol. 14, no. 1, pp. 342–349, January 2018. [Online]. Available: <https://doi.org/10.1109/TII.2016.2637166>
- [3] N. Niu, S. Reddivari, and Z. Chen, “Keeping requirements on track via visual analytics,” in *International Requirements Engineering Conference (RE)*, Rio de Janeiro, Brazil, July 2013, pp. 205–214. [Online]. Available: <https://doi.org/10.1109/RE.2013.6636720>
- [4] M. Alenazi, N. Niu, and J. Savolainen, “A novel approach to tracing safety requirements and state-based design models,” in *International Conference on Software Engineering (ICSE)*, Seoul, South Korea, June-July 2020, pp. 848–860. [Online]. Available: <https://doi.org/10.1145/3377811.3380332>
- [5] W. Wang, F. Dumont, N. Niu, and G. Horton, “Detecting software security vulnerabilities via requirements dependency analysis,” *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1665–1675, May 2022. [Online]. Available: <https://doi.org/10.1109/TSE.2020.3030745>
- [6] A. Mahmoud and N. Niu, “Supporting requirements traceability through refactoring,” in *International Requirements Engineering Conference (RE)*, Rio de Janeiro, Brazil, July 2013, pp. 32–41. [Online]. Available: <https://doi.org/10.1109/RE.2013.6636703>
- [7] N. Niu, W. Wang, and A. Gupta, “Gray links in the use of requirements traceability,” in *International Symposium on Foundations of Software Engineering (FSE)*, Seattle, WA, USA, November 2016, pp. 384–395. [Online]. Available: <https://doi.org/10.1145/2950290.2950354>
- [8] J. Huang, B. Berenbach, S. Clark, R. Settini, and E. Romanova, “Best practices for automated traceability,” *Computer*, vol. 40, no. 6, pp. 27–35, June 2007. [Online]. Available: <https://doi.org/10.1109/MC.2007.195>
- [9] N. Alturayef, J. Hassine, and I. Ahmad, “Machine learning approaches for automated software traceability: A systematic literature review,” *Journal of Systems and Software*, vol. 230, pp. 112 536:1–112 536:38, December 2025. [Online]. Available: <https://doi.org/10.1016/j.jss.2025.112536>
- [10] J. Lin, Y. Liu, Q. Zeng, M. Jiang, and J. Cleland-Huang, “Traceability transformed: Generating more accurate links with pre-trained BERT models,” in *International Conference on Software Engineering (ICSE)*, Madrid, Spain, May 2021, pp. 324–335. [Online]. Available: <https://doi.org/10.1109/ICSE43902.2021.00040>
- [11] F. Trad and A. Chehab, “Retrieval-augmented few-shot prompting versus fine-tuning for code vulnerability detection,” in *International Conference on Foundation and Large Language Models (FLLM)*, Vienna, Austria, November 2025, pp. 615–622. [Online]. Available: <https://doi.org/10.1109/FLLM67465.2025.11391248>
- [12] J. Zhang, Y. Chen, N. Niu, and C. Liu, “A preliminary evaluation of ChatGPT in requirements information retrieval,” *CoRR*, July 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.12562>
- [13] T. Hey, D. Fuchß, J. Keim, and A. Koziol, “Requirements traceability link recovery via retrieval-augmented generation,” in *International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ)*, Barcelona, Spain, April 2025, pp. 381–397. [Online]. Available: https://doi.org/10.1007/978-3-031-88531-0_27
- [14] J. Zhang, J. Zhou, N. Niu, J. Hua, and C. Liu, “Synergistic enhancement of requirement-to-code traceability: A framework combining large language model based data augmentation and an advanced encoder,” *Information Software Technology*, vol. 193, pp. 108 045:1–108 045:20, May 2026. [Online]. Available: <https://doi.org/10.1016/j.infsof.2026.108045>

- [15] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *International Conference on Machine Learning (ICML)*, Virtual Event, July 2021, pp. 12 697–12 706. [Online]. Available: <http://proceedings.mlr.press/v139/zhao21c.html>
- [16] W. Wang, N. Niu, H. Liu, and Z. Niu, "Enhancing automated requirements traceability by resolving polysemy," in *International Requirements Engineering Conference (RE)*, Banff, AB, Canada, August 2018, pp. 40–51. [Online]. Available: <https://doi.org/10.1109/RE.2018.00-53>
- [17] J. Guo, J. Cheng, and J. Cleland-Huang, "Semantically enhanced software traceability using deep learning techniques," in *International Conference on Software Engineering (ICSE)*, Buenos Aires, Argentina, May 2017, pp. 3–14. [Online]. Available: <https://doi.org/10.1109/ICSE.2017.9>
- [18] E. Alor, S. Khatoonabadi, and E. Shihab, "Evaluating the use of LLMs for documentation to code traceability," *CoRR*, June 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2506.16440>
- [19] M. Dahiya, M. Li, G. Horton, T. Scherz, and N. Niu, "Tracing feature tests to textual requirements," in *International Conference on Information Reuse and Integration (IRI)*, San Jose, CA, USA, August 2024, pp. 120–125. [Online]. Available: <https://doi.org/10.1109/IRI62200.2024.00035>
- [20] N. M. Iluru, N. Niu, Y. Yang, and Y. Wang, "A meta-learning approach to generating functional descriptions of graphical user interfaces," in *International Conference on Information Reuse and Integration (IRI)*, San Jose, CA, USA, August 2025, pp. 7–12. [Online]. Available: <https://doi.org/10.1109/IRI66576.2025.00010>
- [21] A. D. Rodriguez, K. R. Dearstyne, and J. Cleland-Huang, "Prompts matter: Insights and strategies for prompt engineering in automated software traceability," in *International Workshop on Software and Systems Traceability (SST)*, Hannover, Germany, September 2023, pp. 455–464. [Online]. Available: <https://doi.org/10.1109/REW57809.2023.00087>
- [22] D. Fuchß, T. Hey, J. Keim, H. Liu, N. Ewald, T. Thirolf, and A. Koziol, "LiSSA: Toward generic traceability link recovery through retrieval-augmented generation," in *International Conference on Software Engineering (ICSE)*, Ottawa, ON, Canada, April-May 2025, pp. 1396–1408. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00186>
- [23] R. Etezadi, S. Abualhaija, C. Arora, and L. C. Briand, "Classifier or prompt: A case study on legal requirements traceability," *Empirical Software Engineering*, vol. 31, no. 4, pp. 85:1–85:47, July 2026. [Online]. Available: <https://doi.org/10.1007/s10664-026-10827-1>
- [24] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the role of demonstrations: What makes in-context learning work?" in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Abu Dhabi, United Arab Emirates, 2022, pp. 11 048–11 064. [Online]. Available: <https://doi.org/10.18653/v1/2022.emnlp-main.759>
- [25] N. Niu, A. Koshoffer, L. Newman, C. Khatwani, C. Samarasinghe, and J. Savolainen, "Advancing repeated research in requirements engineering: A theoretical replication of viewpoint merging," in *International Requirements Engineering Conference (RE)*, Beijing, China, September 2016, pp. 186–195. [Online]. Available: <https://doi.org/10.1109/RE.2016.46>
- [26] C. Khatwani, X. Jin, N. Niu, A. Koshoffer, L. Newman, and J. Savolainen, "Advancing viewpoint merging in requirements engineering: A theoretical replication and explanatory study," *Requirements Engineering*, vol. 22, no. 3, pp. 317–338, September 2017. [Online]. Available: <https://doi.org/10.1007/s00766-017-0271-0>
- [27] M. Dahiya, R. Gill, N. Niu, H. Gudaparthi, and Z. Peng, "Leveraging ChatGPT to predict requirements testability with differential in-context learning," in *International Conference on Information Reuse and Integration (IRI)*, San Jose, CA, USA, August 2024, pp. 170–175. [Online]. Available: <https://doi.org/10.1109/IRI62200.2024.00044>
- [28] A. S. Nyamawe, H. Liu, N. Niu, Q. Umer, and Z. Niu, "Feature requests-based recommendation of software refactorings," *Empirical Software Engineering*, vol. 25, no. 5, pp. 4315–4347, September 2020. [Online]. Available: <https://doi.org/10.1007/s10664-020-09871-2>
- [29] V. Niu, W. Alshammari, N. M. Iluru, P. V. Teeleti, N. Niu, T. Bhowmik, and J. Zhang, "Exploiting vision-language models in GUI reuse," in *International Conference on Software and Systems Reuse (ICSR)*, Ottawa, ON, Canada, April 2025, pp. 21–32. [Online]. Available: <https://doi.org/10.1109/ICSR66718.2025.00009>
- [30] S. Reddivari, Z. Chen, and N. Niu, "ReCVisu: A tool for clustering-based visual exploration of requirements," in *International Requirements Engineering Conference (RE)*, Chicago, IL, USA, September 2012, pp. 327–328. [Online]. Available: <https://doi.org/10.1109/RE.2012.6345828>
- [31] H. Gudaparthi, N. Niu, J. Zhang, and J. Savolainen, "Applying cluster hypothesis to the next release problem," in *International Conference on Information Reuse and Integration (IRI)*, San Jose, CA, USA, August 2024, pp. 258–263. [Online]. Available: <https://doi.org/10.1109/IRI62200.2024.00060>